

Reparación Automática de Software

High Order Mutants - Fault Localization

Mutation Testing

La idea bajo Mutation Testing es emular defectos y evaluar una test suite en cuanto a su capacidad de detectar estos defectos.

Mutation Testing se basa en 4 partes:

1. Un programa P, correcto, escrito en una gramática G.
2. Un conjunto de operadores de mutación, cada operador especifica una regla de variación/modificación sintáctica de una cadena en G.
3. Mutantes, variaciones de P a partir de una modificación sintáctica generada por un operador de mutación.
4. Evaluar cuantos defectos (mutantes) son detectados por la test suite.

¿Cuales son los Requisitos de Testing?

Mutation Score

Dado un programa **P**, un mutante **m** de **P** y un conjunto de test suite **TS** sobre **P** a evaluar, las opciones para **m** son:

1. Al menos un test en **TS** falla para **m** (*), detectando o “matando” a **m**.
2. Ningún test en **TS** falla para **m**, pero **m** contiene un defecto detectable.
3. Ningún test en **TS** falla para **m**, pero **m** no contiene un defecto detectable (**)

(*) Es posible matar un mutante mediante el compilador, el intérprete, o una excepción, estos mutantes suelen considerarse **triviales**.

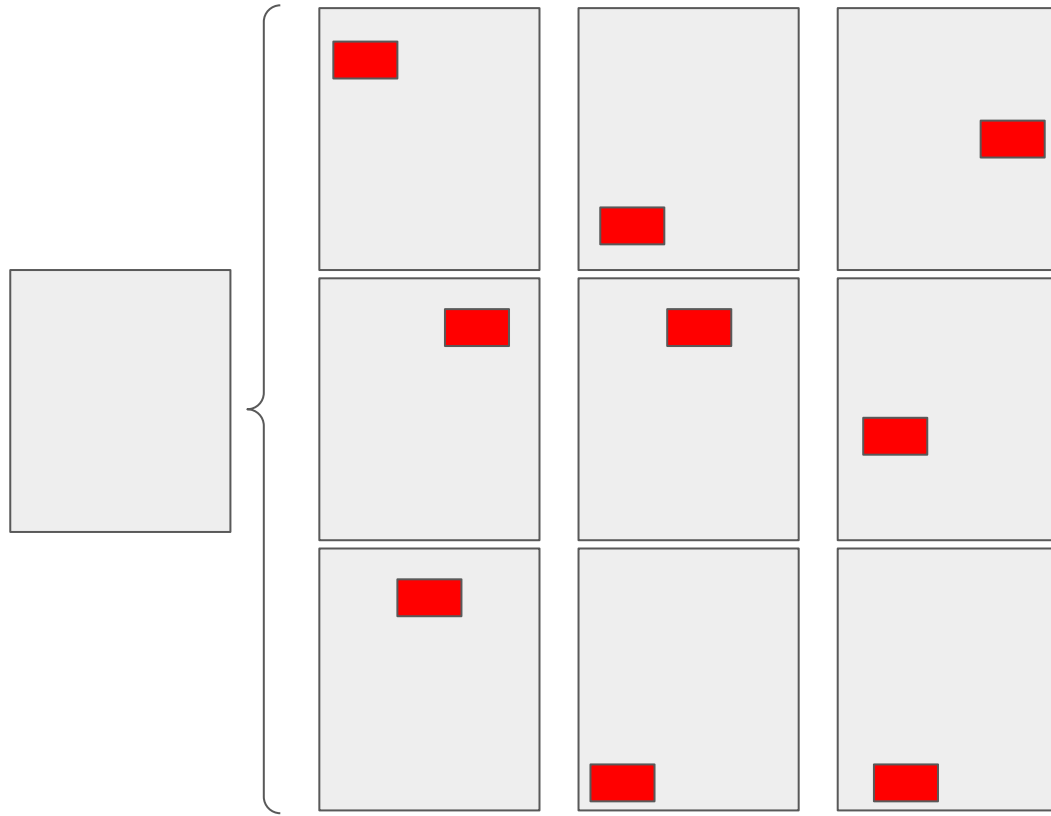
(**) Un ejemplo son los mutantes equivalentes, el mutante es sintácticamente diferente de **P** pero semánticamente es indistinguible.

Mutation Score

Dado un programa **P**, un mutante **m** de **P** y un conjunto de test suite **TS** sobre **P** a evaluar.

De la misma forma en que se calcula cobertura de sentencias, es decir **sentencias cubiertas / sentencias totales**, podemos calcular mutantes **detectados / mutantes totales** (mutation score).

First Order Mutants (FOMs)



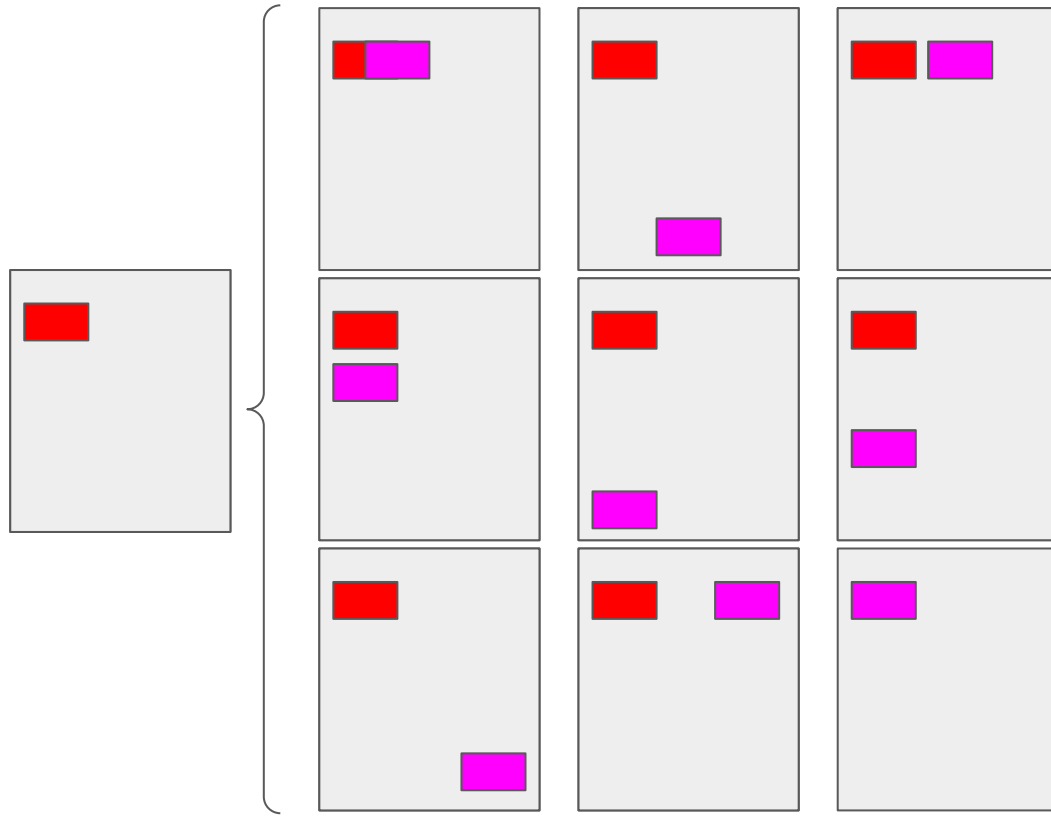
Dado un programa P , se generan versiones P^D (mutantes) con cada una conteniendo una mutación (un cambio sintáctico simple).

Estos mutantes, cada uno con una mutación, se llaman

“Mutantes de Primer Orden”

Una pregunta que se puede hacer es: ¿Si un cambio sintáctico puede representar defectos reales, más de un cambio sintáctico podrían representar defectos reales más sutiles (más difíciles de detectar)?

High Order Mutants (HOMs)



Dado un programa P , se generan versiones P^D (mutantes) con cada una conteniendo n mutación (cada una, un cambio sintáctico simple).

Estos mutantes, cada uno con n mutaciones, se llaman

“Mutantes de Alto Orden”

Un mutante de alto orden se forma a partir de n mutantes de primer orden.

High Order Mutants (HOMs)

La generación de HOMs se puede ver como una iteración de la generación de mutantes:

Generación 0: programa original P

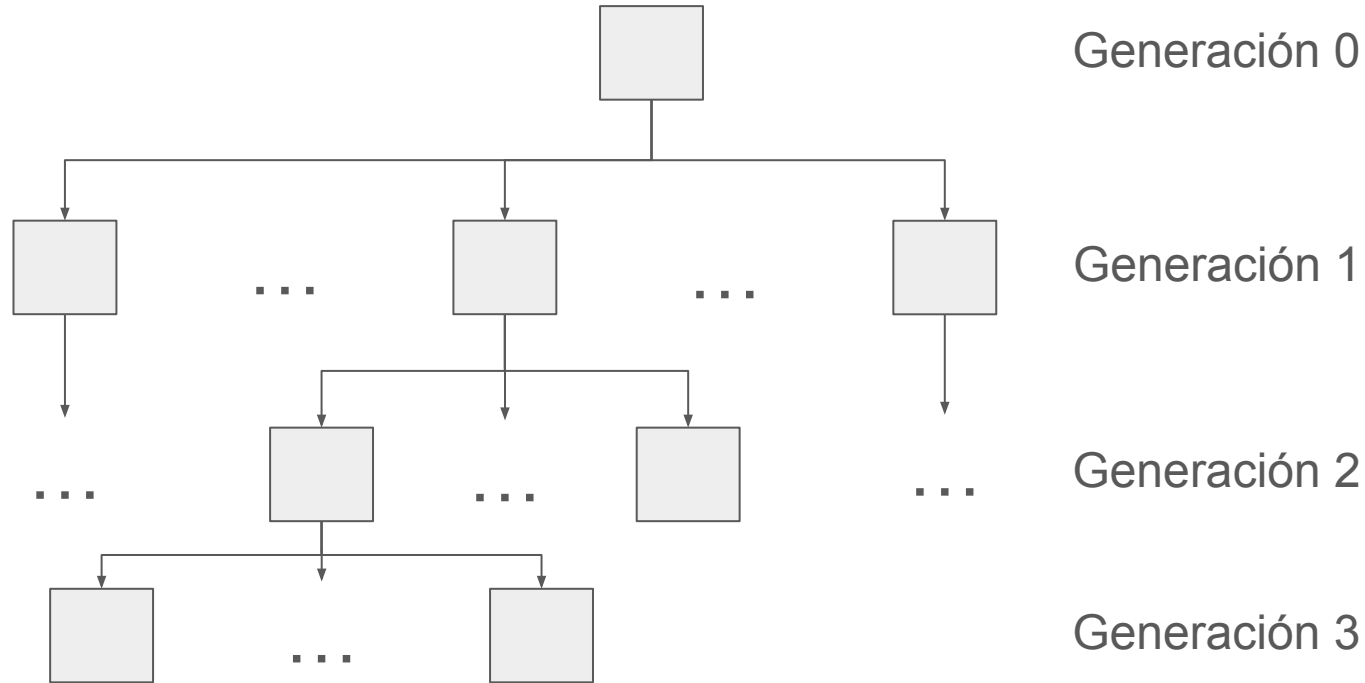
Generación 1: para cada miembro de la Generación 0 generar mutantes (cada uno con una mutación)

Generación 2: para cada miembro de la Generación 1 generar mutantes (cada uno con una mutación nueva).

...

Generación N: para cada miembro de la Generación N - 1 generar mutantes (cada uno con una mutación nueva)

High Order Mutants (HOMs)



High Order Mutants (HOMs)

Ventajas y Desventajas

Ventajas:

- Defectos más sutiles.
- HOM subsumption, dado un mutante de alto orden h proveniente de mutantes de primer orden $f_1, f_2, \dots, f_n \mid h$ subsume los mutantes $f_1, \dots, f_n$ si el conjunto de tests T_h que detectan a h , también detectan cada mutante $f_1, \dots, f_n$. En estos casos, utilizar h (un mutante) evalúa de la misma forma T_h que $f_1, \dots, f_n$.

High Order Mutants (HOMs)

Ventajas y Desventajas

Desventajas:

- Explosión de mutantes, cómo cota inferior (y muy inferior)
 $\text{mutantes}(G_n) = \text{mutantes}(G_1)^n$.
- ???

High Order Mutants (HOMs)

Ventajas y Desventajas

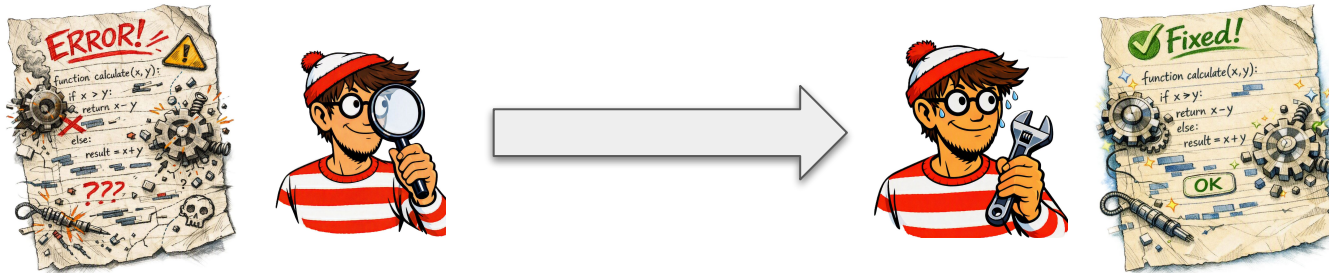
Desventajas:

- Explosión de mutantes, cómo cota inferior (y muy inferior)
 $\text{mutantes}(\mathbf{G}_n) = \text{mutantes}(\mathbf{G}_1)^n$.
- Mutantes repetidas/duplicadas.
- Reversión de mutaciones (puramente sintáctica o semántica).
(Este punto es muy interesante a explorar!)

Fault Localization

Dado un programa defectuoso P_D para el cual existe un conjunto de tests T en el cual existe al menos un test t que falla para P_D el problema de Fault Localization es determinar la ubicación de el o los defectos causantes de que t falle.

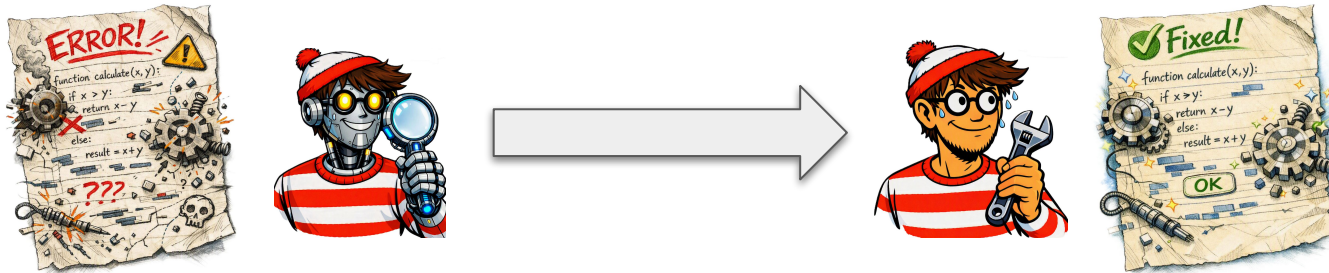
Esta tarea es el proceso de debugging, manualmente es una tarea difícil y que lleva mucho tiempo (entre otros recursos).



Fault Localization

Dado un programa defectuoso P_D para el cual existe un conjunto de tests T en el cual existe al menos un test t que falla para P_D el problema de Fault Localization es determinar la ubicación de el o los defectos causantes de que t falle.

Automatizar esta tarea es una área de investigación de gran interés.



Fault Localization - Spectrum Based FL

Dado un programa defectuoso P_D para el cual existe un conjunto de tests T en el cual existe al menos un test t que falla para P_D el problema de Fault Localization es determinar la ubicación de el o los defectos causantes de que t falle.

Un enfoque es utilizar la información de cobertura dada por los tests para generar un “*índice de sospecha*” para cada sentencia de P_D (aunque podría ser posible utilizar elementos más pequeños que a nivel sentencia).

La idea:

- Por cada test $t \in T$ si t pasa, sumar $+1$ *positivo* a cada sentencia cubierta/ejecutada por t .
- Por cada test $t \in T$ si t falla, sumar $+1$ *negativo* a cada sentencia cubierta/ejecutada por t .
- Calcular por cada sentencia un “*índice de sospecha*” basada en marcas positivas, marcas negativas, tests totales, tests positivos, tests negativos.
- Ordenar las sentencias de manera descendente de acuerdo a su “*índice de sospecha*”.

Fault Localization - Mutation Based FL

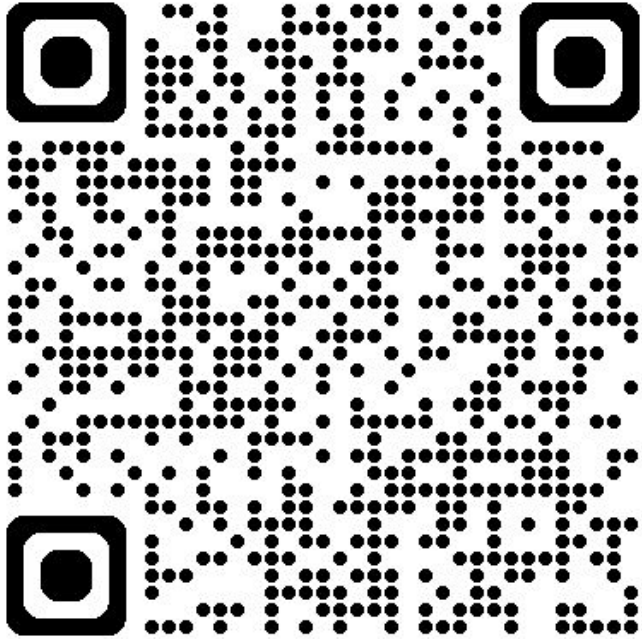
Dado un programa defectuoso P_D para el cual existe un conjunto de tests T en el cual existe al menos un test t que falla para P_D el problema de Fault Localization es determinar la ubicación de el o los defectos causantes de que t falle.

Si se toma un elemento sintáctico en P_D , denominado x , si este elemento x es modificado generando una versión modificada P_{Dx} tal que para todo test $t \in T \mid t$ pasa en P_D y pasa P_{Dx} ó t falla en P_D pero pasa en P_{Dx} .

Es decir, P_{Dx} mantiene el comportamiento correcto de P_D y revierte algún comportamiento incorrecto de P_D .

El elemento x es altamente sospechoso de ser causa del defecto en P_D .

Fault Localization - Spectrum Based FL



FaultLocalization (nombre poco original)

Basado en el trabajo de tesis de Joaquin Heredia

<https://github.com/saiema/FaultLocalization/tree/introclass>

Fórmulas implementadas:

- TARANTULA
- OCHIAI
- JACCARD
- OP2
- BARINEL
- DSTAR